

DIGITALITZACIÓ D'UN TALLER MECÀNIC CONNECTAT

Mòdul 1665 - Digitalització Aplicada als Sectors Productius

Camp	Dada
Nom de l'equip	Víctor Díaz / Equip ASIX 1
Curs	ASIX 1
Professor	David González
Data de lliurament	Maig de 2026
Nom del projecte	Taller Mecànic Connectat

Índex

1. Introducció
2. Anàlisi inicial
3. Planificació i cronograma
4. Model de dades
5. Desenvolupament de la solució
 - 5.1 API Flask
 - 5.2 Web HTML/CSS/JS
 - 5.3 Scripts d'automatització
6. Infraestructura de sistemes amb Docker
7. Proves de funcionament
8. Problemes i solucions
9. Conclusions
10. Annexos

1. Introducció

Aquest projecte consisteix en digitalitzar la gestió bàsica d'un petit taller mecànic. Fins ara el taller treballava amb paper per apuntar clients, vehicles i cites. Amb aquesta solució es crea una aplicació senzilla formada per una pàgina web, una API amb Python Flask, una base de dades MariaDB i contenidors Docker.

L'objectiu principal és que el taller pugui guardar la informació de manera ordenada i que els clients puguin demanar una cita des d'una web. També s'inclouen scripts per fer còpies de seguretat i comprovar que els serveis funcionen correctament.

2. Anàlisi inicial

El client és un taller mecànic petit que necessita una eina simple i fàcil d'utilitzar. No es busca una aplicació molt complexa, sinó una solució realista per a ASIX 1 que compleixi els requisits del projecte.

Necessitat detectada	Solució aplicada
Guardar clients	Taula clients a MariaDB i ruta GET /clients.
Guardar vehicles	Taula vehicles relacionada amb clients i ruta GET /vehicles.
Demanar cites online	Formulari web que envia dades a POST /appointments.
Consultar cites	Ruta GET /appointments i llistat a la web.
Evitar instal·lacions manuals complicades	Docker Compose per aixecar web, API i base de dades.
Fer còpies de seguretat	Script backup.sh amb mysqldump.
Revisar serveis	Script check_services.sh amb curl, docker ps i mysqladmin.

3. Planificació i cronograma

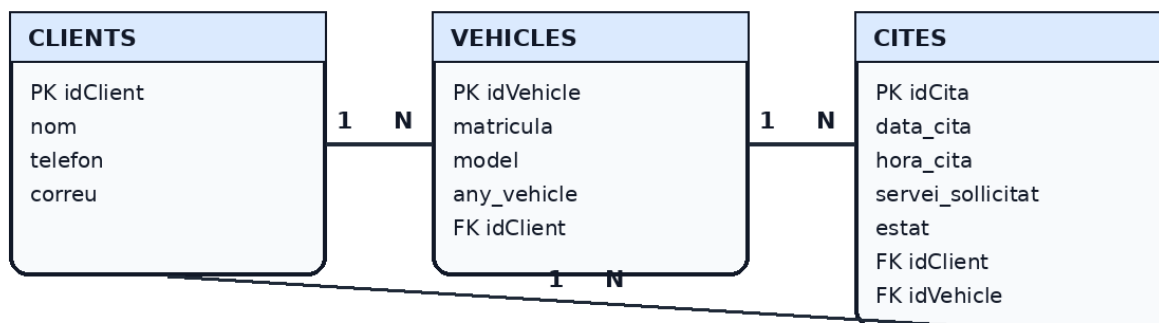
La feina s'ha dividit en fases senzilles. Primer s'ha analitzat què demana el projecte, després s'ha creat la base de dades i finalment s'ha connectat tot amb Docker.

Sessió	Tasques previstes	Tasques realitzades
1	Llegir l'enunciat, definir rols i preparar el model de dades.	Anàlisi inicial i creació de les taules clients, vehicles i cites.
2	Crear l'API Flask i provar rutes bàsiques.	Rutes GET /clients, GET /vehicles, GET /appointments i POST /appointments.
3	Fer la web amb formulari i fetch.	Web amb formulari de cita, llistat de cites i vehicles.
4	Crear scripts i Docker Compose.	backup.sh, check_services.sh, Dockerfile API, Dockerfile web i docker-compose.yml.
5	Revisar, documentar i preparar la demo.	Memòria, README i comprovacions finals.

4. Model de dades

La base de dades es diu taller i està formada per tres taules principals: clients, vehicles i cites. Les relacions són senzilles: un client pot tenir diversos vehicles i cada vehicle pot tenir diverses cites.

Diagrama E/R - Taller Mecànic Connectat



Relacions:

- Un client pot tenir diversos vehicles.
- Un vehicle pot tenir diverses cites.
- Una cita queda associada a un client i a un vehicle.

Taula	Camps principals	Explicació
clients	idClient, nom, telefon, correu	Guarda les dades bàsiques dels clients.
vehicles	idVehicle, matricula, model, any_vehicle, idClient	Guarda els vehicles i els relaciona amb un client.
cites	idCita, data_cita, hora_cita, servei_sollicitat, estat, idClient, idVehicle	Guarda les cites demanades pels clients.

Script SQL principal:

```
CREATE TABLE clients (...);  
CREATE TABLE vehicles (... FOREIGN KEY (idClient) REFERENCES clients(idClient));  
CREATE TABLE cites (... FOREIGN KEY (idClient) REFERENCES clients(idClient), FOREIGN KEY (idVehicle)  
REFERENCES vehicles(idVehicle));
```

5. Desenvolupament de la solució

5.1 API

L'API s'ha fet amb Python i Flask. La comunicació amb la web es fa amb JSON. La API connecta amb MariaDB utilitzant mysql-connector-python.

Ruta	Mètode	Funció
/	GET	Comprovar que l'API està activa.
/health	GET	Comprovar API i base de dades.
/clients	GET	Llistar clients.
/vehicles	GET	Llistar vehicles amb el nom del client.
/appointments	GET	Llistar cites.
/appointments	POST	Crear una cita nova.

```
curl http://localhost:5000/vehicles
curl http://localhost:5000/appointments
curl http://localhost:5000/health
```

5.2 Web

La web està feta amb HTML, CSS i JavaScript bàsic. Té un formulari per demanar cita i dues zones de consulta: una per veure les cites registrades i una altra per veure els vehicles.

El JavaScript utilitza fetch() per enviar i rebre dades de l'API. Quan es crea una cita, la web mostra un missatge i actualitza el llistat de cites.

Fitxer	Funció
web/index.html	Estructura de la pàgina i formulari.
web/css/style.css	Disseny visual i adaptació a mòbil.
web/js/script.js	Peticions fetch a l'API i actualització de la pantalla.

5.3 Scripts

S'han creat dos scripts simples en Bash per automatitzar tasques de sistemes.

Script	Comanda	Què fa
backup.sh	bash scripts/backup.sh	Fa un mysqldump de la base de dades taller i el guarda a backups/.
check_services.sh	bash scripts/check_services.sh	Comprova els contenidors, l'API, MariaDB i la web.

6. Infraestructura de sistemes amb Docker

La solució es desplega amb Docker Compose. S'han creat tres contenidors: un per MariaDB, un per l'API Flask i un per la web amb Nginx.

Servei	Contenidor	Port	Nom intern
Base de dades	taller_db	3306 intern	db.taller.local
API Flask	taller_api	5000	api.taller.local
Web Nginx	taller_web	8080:80	web.taller.local

Tots els serveis estan connectats a la xarxa Docker taller_net. Això permet que l'API pugui arribar a la base de dades utilitzant el nom db.

```
docker-compose up --build
docker-compose down
docker ps
```

7. Proves de funcionament

Per comprovar el projecte, primer s'aixequen els contenidors i després es proven la web, l'API i els scripts.

Prova	Resultat esperat
docker-compose up --build	S'aixequen els contenidors taller_db, taller_api i taller_web.
http://localhost:8080	Es mostra la web del taller.
http://localhost:5000/vehicles	Es veuen els vehicles en format JSON.
http://localhost:5000/appointments	Es veuen les cites en format JSON.
Formulari web	Es crea una cita nova i apareix al llistat.
bash scripts/backup.sh	Es crea un fitxer .sql dins de backups/.
bash scripts/check_services.sh	Mostra que web, API i MariaDB funcionen.

Captures recomanades per entregar: docker ps, web funcionant, GET /vehicles, GET /appointments, creació d'una cita i execució dels scripts.

8. Problemes i solucions

Problema	Solució aplicada
La web podia tenir problemes de CORS en cridar l'API.	S'ha afegit Flask-CORS a l'API.
La base de dades podia no estar preparada quan arrenca l'API.	S'ha afegit una espera curta inicial i depends_on al docker-compose.yml.
Els contenidors necessiten comunicar-se entre ells.	S'ha creat una xarxa Docker pròpia: taller_net.
Calia deixar dades de prova.	S'han afegit INSERT inicials al db_schema.sql.
Calia fer còpies de seguretat.	S'ha creat backup.sh utilitzant mysqldump.

9. Conclusions

Amb aquest projecte s'ha après a crear una solució completa de digitalització amb diferents parts connectades entre elles. La web permet demanar cites, l'API centralitza les dades, MariaDB les guarda i Docker facilita el desplegament.

També s'ha practicat l'ús de scripts de sistemes, rutes API, consultes SQL, fitxers Dockerfile i docker-compose.yml. Com a millores futures es podria afegir un login d'administrador, validació més avançada de dades, edició de cites i una pàgina privada per al personal del taller.

10. Annexos

Fitxer o carpeta	Contingut
api/app.py	Codi de l'API Flask.
api/requirements.txt	Llibreries de Python.
web/index.html	Pàgina principal.
web/css/style.css	Estils de la web.
web/js/script.js	JavaScript amb fetch.
scripts/backup.sh	Script de còpia de seguretat.
scripts/check_services.sh	Script de comprovació de serveis.
db_schema.sql	Creació de la base de dades i dades inicials.
docker-compose.yml	Desplegament dels contenidors.
docs/db_erschema.png	Diagrama E/R del projecte.

Comandes principals per a la demo:

```
docker-compose up --build
# Web: http://localhost:8080
# API: http://localhost:5000/vehicles
bash scripts/backup.sh
bash scripts/check_services.sh
```